Topic 4: Files

Learning Outcomes:
(a) Identify the use of input/output files using CSV and text files

Topic 4: Files

Table of content

1. Why use file
2. Type of files
3. Working with .txt
    a. Open file
    b. Read
    c. Write
    d. Close
4. Working with .csv
    a. Open file
    b. Read
    c. Write
    d. Close

1. Why use Files?

Learning about text files in Python is important because:

- Data Persistent/storage : Data stored in memory will be deleted when the program terminate
- Data Exchange: They're a common format for exchanging data between programs.
- File Handling Skills: Working with text files teaches essential file handling operations.
- Configuration: Many applications use text files for configuration settings.
- Data Processing: Text files are often used as input sources for data processing tasks.
- Compatibility: They're platform-independent and easily transferable between systems and applications.

2. Type of File in Python

There are 2 type of files in python

- **Text File (our focus)** : Text file usually we use to store character data. For example, test.txt

- **Binary Files** is used to store binary data such as images, video files, audio files, etc.

What is text File ?

- A file that contains human readable character
- No embedded formatting commands (like **bold** or *italic*) or binary data
- Data represented in sequences of (ASCII) character organized into lines
- **Typically** terminated by **new line character (\n)**
- Common text file format is as shown **Figure 1**

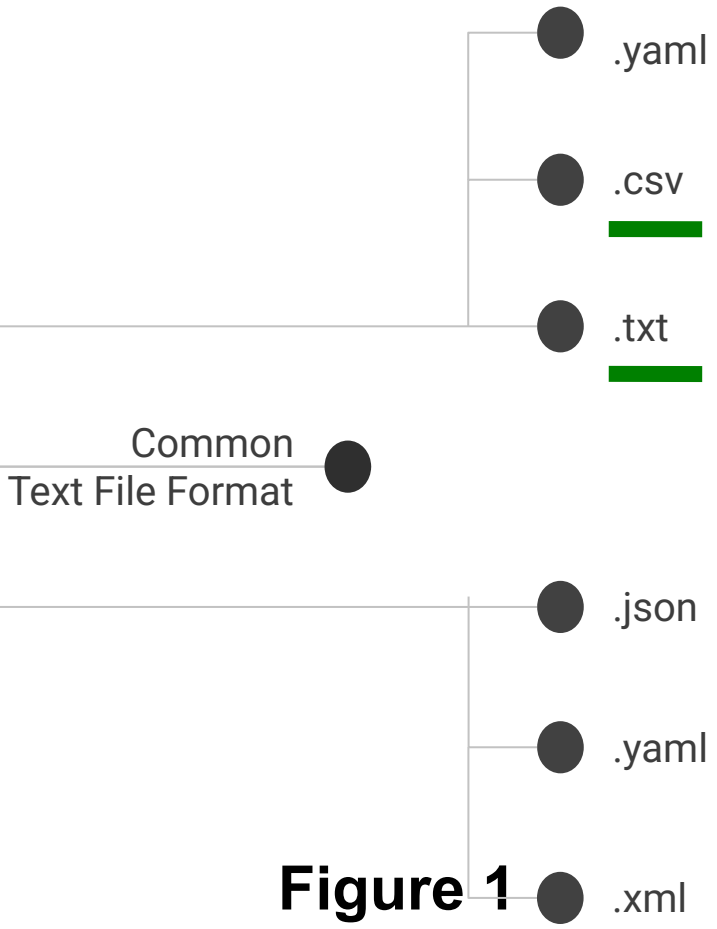


Figure 1

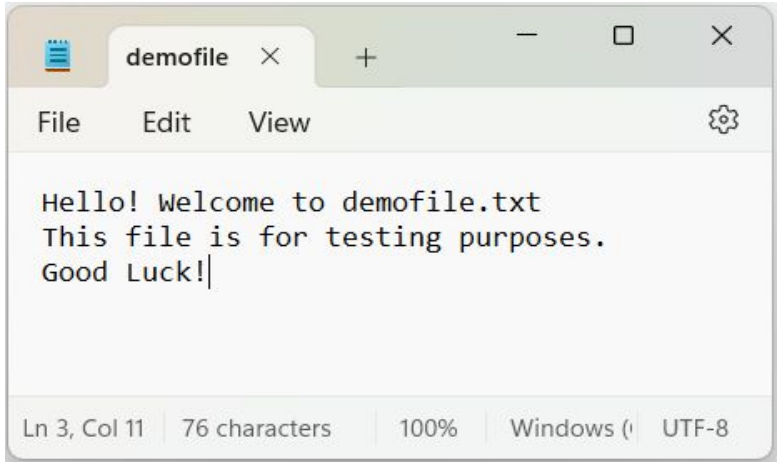
Reference [What is csv file]

What is .txt and .csv File

.txt

- Pure text document that only contains unformatted or unstructured text.
- File is recognized by any text editing or word processing program.

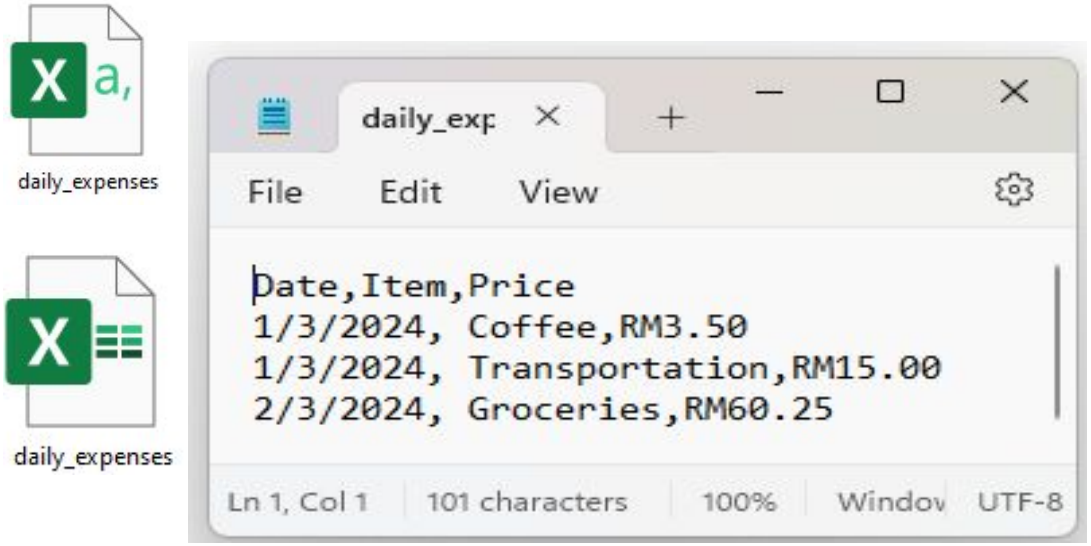
demofile.txt



.csv

- Simplest **spreadsheet/database file**.
- can be open with spreadsheet software like Excel and Google Sheet

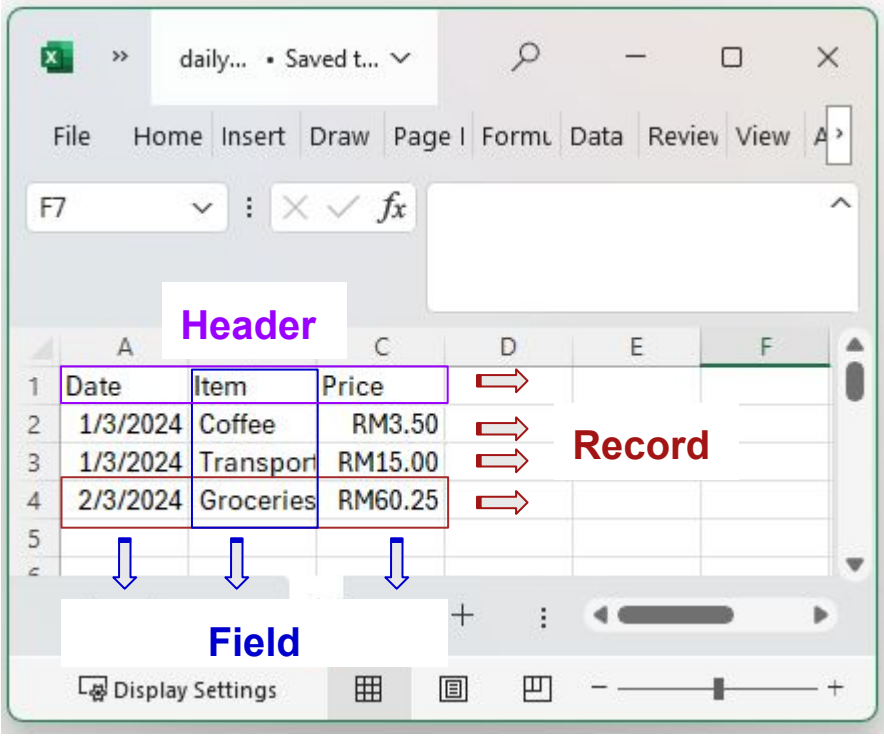
daily_expenses



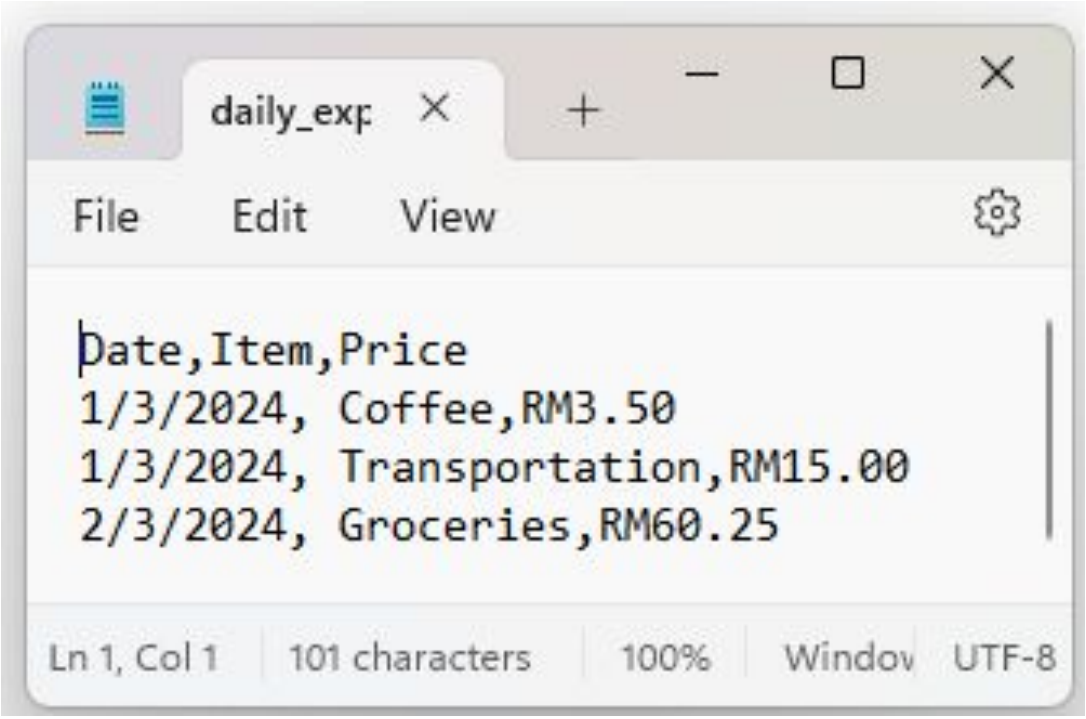
Structure of .csv File

- **row** is a record
- **column** is a field within the record.
- **the first row** is a header
- **values is separated by comma**, hence the name comma separated value ***(csv)***

open in excel



open in notepad



3. File Operation for .txt and .csv file

Basic file operations

1. Open File

2. Read File

3. Write to File

4. Close File

Python provides built-in functions and external modules(**csv**) to support these operations

3. Working with .txt file

.txt File

.txt Python File Operation Functions & Methods

open file

```
f= open(path_to_file,mode)
```

read file

```
f.read()        # read entire content of the file as a STRING
f.readline() # read a single line from a file as a STRING
f.readlines()# read all lines in the file and return them as a LIST of STRING
```

write to file

```
f.write()        # write a single line to a file
f.writelines() # write a list of string to a file
```

close file

```
f.close()        # close the file
```

1. Opening File Syntax

```
f= open(path_to_file,mode)
```

The mode you pick decides what action you can do with the file and behaviour of the file.

`'path/to/txt/file/expenses.txt'`

r - read	r+ - read/write
a - append	a+ - append/read
w - write	w+ - write/read

Opening Mode Description,Action and Behavior

Mode	Description	Action	Behavior
'w'	Write mode	Creates or clears file to zero length	Overwrites existing content or creates new file
'w+'	Read/write mode	Reads and writes, creates file if not exist	Overwrites existing content or creates new file, allows reading and writing
'r'	Read mode	Reads file contents	Gives an error if the file does not exist, read-only
'r+'	Read/write mode	Reads and writes	Reads and writes without altering existing content, gives an error if the file does not exist
'a'	Append mode	Writes at the end, adds to current content	Adds to existing content or creates a new file if it doesn't exist
'a+'	Read/append mode	Reads and adds to file	Reads and appends to existing content or creates a new file if it doesn't exist

Opening File Syntax in Different File Location

File location	Opening File Syntax
Same folder	`f=open('demo.txt',w+)`
Local File **(e.g. inside Documents)**	`f=open('c:/User/user/Documents/project_folder/demo.csv',w+)`
Google Drive (Google Colab)	`f=open('/content/drive/MyDrive/project_folder/demo.txt',w+)`

Mode 'w+' , The file will be created if the file does not exist.

File Does Not Exist Error

Opening File Syntax

```
f=open('demofiles.txt', 'r')
print(f.readlines())
```

Error Output

```
Traceback (most recent call last):
    File "c:/User/user/Documents/filenotexist.py", line 9, in <module>
        f=open('demofiles.txt', 'r')
FileNotFoundError: [Errno 2] No such file or directory: 'demofiles.txt'
```

***ERROR always happened cause by either wrong directory OR file does not exist**

2. Reading File Syntax

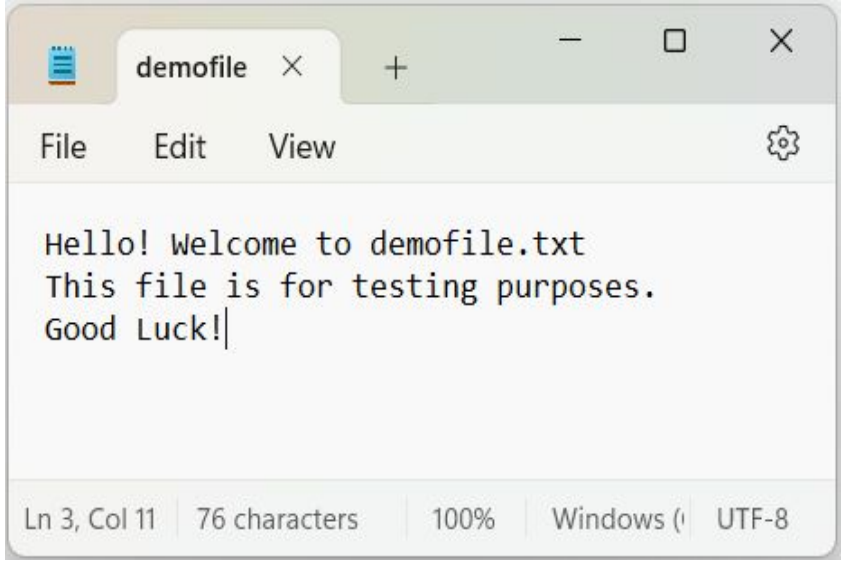
```
f=open(path/to/csv/file/expenses.txt,w+)

f.read()
```

.txt file can be read using the built-in read function

2(a) Reading Entire content of file

Input file :
demofile.txt




```
f = open("demofile.text", "r")
text = f.read()
print(f.read())
print(type(text))
```

output

```
Hello! Welcome to demofile.txt
This file is for testing purposes.
Good Luck!
<class 'str'>
```

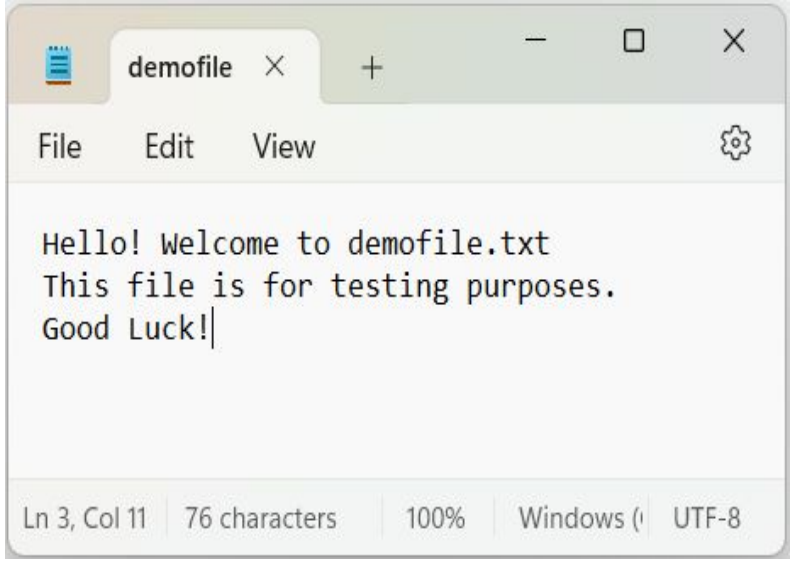
2(b) Reading a Single Line

It reads one line from the file each time it is called. Because it only called one, only one line is read.

```
f = open("demofile.txt", "r")

print(f.readline())
```

Input file:
demofile.txt



output

```
Hello! Welcome to demofile.txt
```

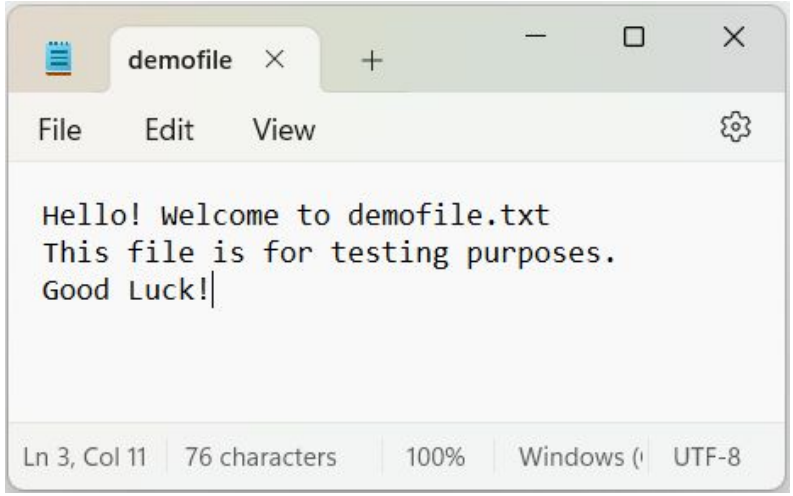
2(b) Reading a Single Line (contn')

It reads one line from the file each time it is called, the program output first and second line as the method readline() is called twice

```
f = open("demofile.txt", "r")

print(f.readline())
print(f.readline())
```

demofile.txt



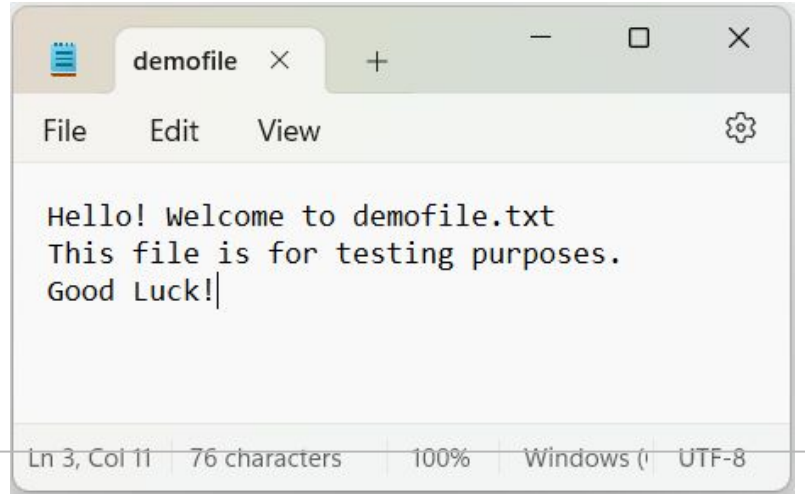
output

```
Hello! Welcome to demofile.txt
This file is for testing purposes.
```

2(c) Reading all Lines

It reads all line, and output a list of string, each element represent a line in the file.

demofile.txt




```
f = open("demofile.txt", "r")

print(f.readlines())
```

output

```
['Hello! Welcome to demofile.txt\n', 'This file is for testing purposes.\n', 'Good Luck!']
```

2(c)(1.0) Reading all Lines - from google colab

We may come across error if your input TEXT file contains unacceptable encoding style(refer to the error message). Make sure input file contains TEXT only, no funny characters.

```
[1] from google.colab import files
    uploaded = files.upload()
```

Choose Files assignment1-string-list.txt

- **assignment1-string-list.txt**(text/plain) - 1271 bytes, last modified: 11/7/2024 - 100% done

```
Saving assignment1-string-list.txt to assignment1-string-list.txt
```

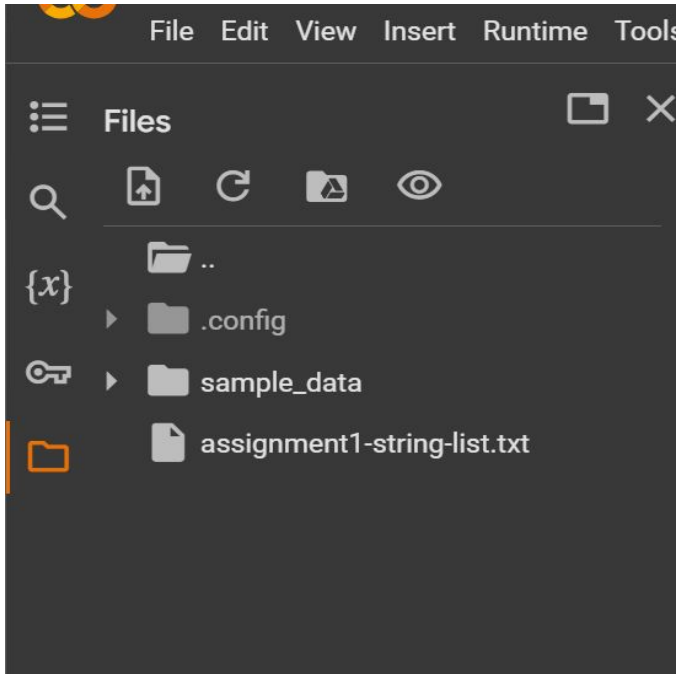
```
f = open('assignment1-string-list.txt', 'r')
print(f.readlines())
```

```
---------------------------------------------------------------------------
UnicodeDecodeError                        Traceback (most recent call last)
<ipython-input-3-4f05ce920721> in <cell line: 16>()
     14
     15 f = open('assignment1-string-list.txt', 'r')
---> 16 print(f.readlines())
     17
     18 #mylist =[]

/usr/lib/python3.10/codecs.py in decode(self, input, final)
    320         # decode input (taking the buffer into account)
    321         data = self.buffer + input
--> 322         (result, consumed) = self._buffer_decode(data, self.errors, final)
    323         # keep undecoded input until the next call
    324         self.buffer = data[consumed:]

UnicodeDecodeError: 'utf-8' codec can't decode byte 0x96 in position 20: invalid start byte
```

File location in colab



2(c)(2.1) Reading all content - from google colab

read(), readline(), readlines()

NOTE: return values are string

```
f = open('/content/input1-ina.txt','r')
text1 = f.read()

print(type(text1),len(text1),text1)
```

```
<class 'str'> 30 Saya ada kucing
Kucing hitam.
```

2(c)(2.2) Reading a line of the content - from google colab

read(), **readline()**, readlines()

NOTE: Return values are string

```
f = open('/content/input1-ina.txt','r')
text1 = f.readline()

print(type(text1),len(text1),text1)
```

```
<class 'str'> 16 Saya ada kucing
```

2(c)(2.3) Reading all content - from google colab

read(),readline(), **readlines()**

Return value is a list.

```
f = open('/content/input1-ina.txt','r')
text1 = f.readlines()

print(type(text1),len(text1),text1)
```

```
<class 'list'> 2 ['Saya ada kucing\n', 'Kucing hitam.\n']
```

2(c)(3.0) Reading all Lines - from google colab

We can use “with …. “ as well

```
from google.colab import drive
drive.mount('/content/drive')

with open('/content/drive/MyDrive/Colab Notebooks/Network diagram.txt', 'r') as f:
    text_content = f.read()

print(text_content)
```

1. mount drive to google colab
2. paste path directory
3. read function

3. Write File Syntax

The `write()` method writes a specified text to the file.

Where the specified text will be inserted depends on the **open file mode** and stream position.

`"a"`: The text will be inserted at the current file stream position, default at the end of the file.

`"w"`: The file will be emptied before the text will be inserted at the current file stream position, default 0.

3. Write File Syntax

```
f=open(path/to/csv/file/expenses.txt,w+)

f.write('content to write\n')

or

f.write('content to write'+'\n')
```

\n is added to the end of the string to ensure the next string start at a new line.

3(a) Write a Single String to File

- **write** method adds a single line to the file.
- you should end the string with \n.
- which indicate end of one line and the beginning of the next line.
- File is close after the program end.

```python
# Open file for writing
file = open('output.txt', 'w')
file.write('Hello, world!\n')
file.write('This is a test.\n')
file.write('Writing strings to a file.\n')
file.close()

# Open file for reading
file = open('output.txt', 'r')
content = file.read()
file.close()

# Display the content
print(content)
```

output

```
Hello, world!
This is a test.
Writing strings to a file.
```

3(a) Write a Single String to File (cont')

- Writing a **list** to a file can also be accomplished using write() method inside a for loop.
- The loop iterates over the list and writes each element to the file
- File is close after the program end.

```python
# Open file for writing
file = open('output.txt', 'w')
lines = ['Hello, world!\n', 'This is a test.\n', 'Writing strings to a file.\n']
for line in lines:
    file.write(line)
file.close()

# Reopen file for reading
file = open('output.txt', 'r')
# Read the content
content = file.read()
file.close()

# Display the content
print(content)
```

output

```
Hello, world!
This is a test.
Writing strings to a file.
```

3(b) Write list of String to File

- A more concise and efficient way is to use **writelines**() method.

```
# Open file for writing
file = open('output.txt', 'w')

# List of strings
lines = ['Hello, world!\n', 'This is a test.\n', 'Writing strings to a file.\n']

# Write all strings from the list to the file
file.writelines(lines)

# Close the file
file.close()
```

output

```
Hello, world!
This is a test.
Writing strings to a file.
```

4. Close File

```
file.close()
```

In Python, it's important to close files after you've finished working with them to release system resources

Topic 4: Files

Learning Outcomes:
(a) Identify the use of input/output files using CSV and text files

Topic 4: Files

Table of content

1. Why use file
2. Type of files
3. Working with .txt
    a. Open file
    b. Read
    c. Write
    d. Close
4. Working with .csv
    a. Open file
    b. Read
    c. Write
    d. Close

4. Working with .csv file

.CSV File

.csv Python File Operation Function & Methods

Python csv module is used handle csv module, import is required

```
import csv
```

1. open file

```
f = open(path_to_file,mode,newline="")
```

2. read file

```
reader=csv.reader(f)    # read entire content of file as an OBJECT
                        # iterate over reader to read the content
```

.csv Python File Operation Function & Methods (contn')

3. write to file

```
writer=csv.writer(f)
writer.writerow()          # write a single row/record to a file
writer.writerows()         # write multiple rows/records to a file
```

4. close file

```
f.close()                  # close the file
```

1. Opening File Syntax

prevent extra blank lines between rows in your CSV file

```
f= open(path_to_file,mode,newline="")
```

'path/to/csv/file/expenses.csv'

The mode you pick decides what action you can do with the file and behaviour of the file.

r - read	r+ - read/write
a - append	a+ - append/read
w - write	w+ - write/read

Adding the `newline=''` parameter into the `open` function ensures that newline characters are handled consistently and correctly by the `csv` module, regardless of the operating system's conventions

Opening Mode Description,Action and Behavior

Mode	Description	Action	Behavior
'w'	Write mode	Creates or clears file to zero length	Overwrites existing content or creates new file
'w+'	Read/write mode	Reads and writes, creates file if not exist	Overwrites existing content or creates new file, allows reading and writing
'r'	Read mode	Reads file contents	Gives an error if the file does not exist, read-only
'r+'	Read/write mode	Reads and writes	Reads and writes without altering existing content, gives an error if the file does not exist
'a'	Append mode	Writes at the end, adds to current content	Adds to existing content or creates a new file if it doesn't exist
'a+'	Read/append mode	Reads and adds to file	Reads and appends to existing content or creates a new file if it doesn't exist

Opening File Syntax in Different File Location

File location	Opening File Syntax
Same folder	`f=open('demo.csv','w+',newline="")`
Local File **(e.g. inside Documents)**	`f=open('c:/User/user/Documents/project_folder/demo.csv',w+)`
Google Drive (Google Colab)	`f=open('/content/drive/MyDrive/project_folder/demo.csv',w+)`

Mode 'w+' , The file will be created if the file does not exist.

2. Reading File Syntax

```
import csv

f=open(path/to/csv/file/expenses.csv,'w+',newline="")

reader=csv.reader(f)
```

.csv file require the import of csv module and creation of reader object to read .csv file

2(a) Reading content of File

- Read the entire content of file
- Close the file after the program end.

daily_expenses.csv

	A	B	C
1	date	description	amount
2	2024-03-01	Coffee	3.5
3	2024-03-01	Transportation	15
4	2024-03-02	Groceries	60.25
5	2024-04-03	Prepaid	30

```python
# importing the csv module
import csv

# Store the file path in csv variable
csv_path='/content/drive/MyDrive/file_handling/daily_expenses.csv'

#opening the file for reading only
f=open(csv_path,'r')
```

```python
# crating reader object
# the contents of file is stored as list of string
reader=csv.reader(f)
```

```python
# iterate the list to print out the element store in the list

for element in reader:
  print(element)

f.close()
```

```
['date', 'description', 'amount']
['2024-03-01', 'Coffee', '3.50']
['2024-03-01', 'Transportation', '15.00']
['2024-03-02', 'Groceries', '60.25']
['2024-04-03', 'Prepaid', '30.00']
```

2(b) Reading content of File and save in a list

- Use list method **extend**(), to save the content into the content list.

- Data is saved in a list for easy manipulation

```python
# Creating an empty list to store the content of the file
content=[]

# Opening the file with reading mode
f=open(csv_path,'r')

# Creating reader object
reader= csv.reader(f)

# Create list form reader object
content.extend(reader)

# Close back the file
f.close()
```

```python
for element in content:
  print(element)
```

```
['date', 'description', 'amount']
['2024-03-01', 'Coffee', '3.50']
['2024-03-01', 'Transportation', '15.00']
['2024-03-02', 'Groceries', '60.25']
['2024-04-03', 'Prepaid', '30.00']
```

3. Write File Syntax

```
import csv

f=open(expenses.txt,w+,newline="")

writer=csv.writer(f)

writer.writerow([field_1,field_2,...])
```

writer object

3(a) Open file with w+ mode and using writerow()

- w+ open mode, create the file if the file does not exist

- w+ overwrites the content of the file if the file already existed.

```
#import csv module
import csv

# patht to file
csv_path='/content/drive/MyDrive/file_handling/test_1.csv'

# opening the with write mode, test_1 content will be overwritten
# test_1 will be created if the file does not exist
f=open(csv_path,'w+',newline='')

#creating writer object
writer=csv.writer(f)

#create the row/record to write to file
record=['name','gender','height']

#write  record to file
writer.writerow(record)

#close the file
f.close()
```

```
# patht to file
csv_path='/content/drive/MyDrive/file_handling/test_1.csv'

# Create empty list
content=[]

# Open the file
f=open(csv_path,'r+')
reader=csv.reader(f)

content.extend(reader)

f.close()

print(content)
```

```
[['name', 'gender', 'height']]
```

3(a) Open file with a+ mode and using writerow()

- Writing a list to a file can also be accomplished using writerow() method inside a for loop.
- The loop iterates over the list and writes each element to the file
- a+ mode, will append the record into the file

```python
import csv

# patht to file
csv_path='/content/drive/MyDrive/file_handling/test_1.csv'

# opening file
f=open(csv_path,'a+',newline='')

# create writer object
writer=csv.writer(f)

# record to write to file
records=[['ali','m','170'],['mariam','f','150'],['abu','m','175']]

# write the records to file
for record in records:
  writer.writerow(record)

#close the file
f.close()
```

```python
# patht to file
csv_path='/content/drive/MyDrive/file_handling/test_1.csv'

# Create empty list
content=[]

# Open the file
f=open(csv_path,'r+')
reader=csv.reader(f)

content.extend(reader)

f.close()

print(content)
```

```
[['name', 'gender', 'height'], ['ali', 'm', '170'], ['mariam', 'f', '150'], ['abu', 'm', '175']]
```

3(b) Open file with a+ mode and using writerows()

- Opening the file with a+ mode, the new content will be appended to the existing content.

- A more concise and efficient way to write list to the file in by using writerows() method

```python
import csv

# patht to file
csv_path='/content/drive/MyDrive/file_handling/test_1.csv'

# opening file
f=open(csv_path,'a+',newline='')

# create writer object
writer=csv.writer(f)

# record to write to file
records=[['ali','m','170'],['mariam','f','150'],['abu','m','175']]

# write the records to file
writer.writerows(records)

#close the file
f.close()
```

```python
# patht to file
csv_path='/content/drive/MyDrive/file_handling/test_1.csv'

# Create empty list
content=[]

# Open the file
f=open(csv_path,'r+')
reader=csv.reader(f)

content.extend(reader)

f.close()

print(content)
```

```
[['name', 'gender', 'height'], ['ali', 'm', '170'], ['mariam', 'f', '150'], ['abu', 'm', '175']]
```

MAKE SURE TO **close()** THE FILE AFTER YOU ARE DONE PROCESSING

4. Close File

```
file.close()
```

In Python, it's important to close files after you've finished working with them to release system resources